

Лабораторная работа №1

Окружение, git и синтаксис Python.
Задача «Миллиард строк»

Содержание

1	Общие положения	2
1.1	Цель работы	2
2	Рабочее окружение	2
2.1	Интерпретатор	2
2.2	Пакеты и виртуальные окружения	2
2.3	Менеджер проектов uv	3
2.4	Git и GitHub	4
3	Синтаксис языка	5
3.1	Структура программы	5
3.2	Переменные, объекты и ссылки	6
3.3	Встроенные типы	6
3.4	Строки	7
3.5	Коллекции	7
3.6	Управляющие конструкции	7
3.7	Генераторы коллекций	8
3.8	Функции	8
3.9	Исключения	9
3.10	Работа с файлами	9
3.11	Соответствие конструкций C++ и Python	10
4	Задание	10
4.1	Часть 1. Создание проекта	10
4.2	Часть 2. Задача «Миллиард строк»	11
5	Порядок сдачи	13

1 Общие положения

1.1 Цель работы

Лабораторная работа предназначена для студентов, освоивших на первом курсе язык C++ и основы объектно-ориентированного программирования. Цель работы — перенос имеющихся знаний на язык Python и изучение его принципиальных отличий от C++. По итогам работы студент должен:

- уметь настраивать рабочее окружение: интерпретатор, виртуальное окружение, менеджер проектов uv;
- уметь вести историю проекта в git и публиковать репозиторий на GitHub;
- владеть базовым синтаксисом языка, встроенными типами данных и работой с файлами;
- уметь решать задачу обработки большого текстового файла средствами стандартной библиотеки.

2 Рабочее окружение

2.1 Интерпретатор

Программа на C++ проходит путь *исходный код* → *компилятор* → *машинный код* → *запуск*. В основной реализации Python (CPython) отдельного этапа сборки нет: интерпретатор python компилирует исходный код в байт-код и сразу исполняет его на собственной виртуальной машине. Как следствие, ошибки, которые в C++ выявляются при компиляции (например, опечатка в имени переменной), в Python проявляются только при выполнении соответствующей строки.

2.2 Пакеты и виртуальные окружения

Сторонние библиотеки устанавливаются менеджером пакетов pip из репозитория PyPI. Установка пакетов в системный интерпретатор приводит к конфликтам: разным проектам могут требоваться несовместимые версии одной библиотеки, а на macOS и Linux системный Python используется самой операционной системой.

Для изоляции проектов применяется **виртуальное окружение** (venv) — каталог внутри проекта, содержащий ссылку на интерпретатор и собственный набор установленных пакетов.

```
$ python3 -m venv .venv          # создание окружения в каталоге .venv
$ source .venv/bin/activate      # активация (macOS / Linux)
> .venv\Scripts\activate         # активация (Windows)
(.venv) $ which python           # .venv/bin/python
(.venv) $ pip install requests   # пакет устанавливается только в .venv
(.venv) $ pip freeze > requirements.txt
(.venv) $ deactivate
```

Такой подход требует ручной активации окружения, ручного ведения списка зависимостей и отдельной установки нужной версии интерпретатора. В курсе вместо него используется менеджер проектов uv.

2.3 Менеджер проектов uv

uv — менеджер проектов для Python, который устанавливает интерпретаторы требуемых версий, создаёт виртуальные окружения, управляет зависимостями и запускает код. По сути он автоматизирует действия, выполняемые вручную с помощью `venv` и `pip`.

Установка

```
# macOS / Linux
$ curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows (PowerShell)
> powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"

$ uv --version
```

Создание проекта

```
$ uv python install 3.13           # установка интерпретатора Python 3.13
$ uv init --python 3.13 lab1       # создание проекта в каталоге lab1
$ cd lab1
$ tree -a -I .git
.
├── .gitignore
├── .python-version                # версия Python для проекта
├── README.md
├── main.py
└── pyproject.toml                # описание проекта и зависимостей
```

Файл `pyproject.toml` выполняет роль, аналогичную `CMakeLists.txt`: в нём указываются имя проекта, требуемая версия Python и список зависимостей.

```
[project]
name = "lab1"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = []
```

Запуск и зависимости

```
$ uv run main.py                  # создаёт .venv при необходимости и запускает скрипт
$ uv add requests                 # добавление зависимости в pyproject.toml и установка
$ uv remove requests              # удаление зависимости
$ uv sync                         # синхронизация .venv с pyproject.toml и uv.lock
```

После первого запуска в проекте появляются каталог `.venv` и файл `uv.lock`. В `lock`-файле фиксируются точные версии всех зависимостей, включая транзитивные, что обеспечивает одинаковое окружение у всех, кто работает с репозиторием.

Редактор

Для работы подходят VS Code с расширением *Python* (Microsoft) или PyCharm. В настройках редактора в качестве интерпретатора необходимо выбрать интерпретатор из каталога `.venv` проекта (в VS Code — команда *Python: Select Interpreter*); в противном случае подсказки и запуск будут использовать другое окружение.

2.4 Git и GitHub

Все лабораторные работы курса сдаются через git-репозиторий; при проверке оценивается как итоговый код, так и история его изменений. Git — система контроля версий, которая хранит последовательность снимков проекта (*коммитов*) и позволяет просматривать изменения и возвращаться к любому из сохранённых состояний. GitHub — сервис для размещения git-репозитория, доступных по ссылке.

Установка и первичная настройка

Git устанавливается с сайта <https://git-scm.com> (на macOS также через `xcode-select --install` или `brew install git`). Перед началом работы указываются имя и адрес электронной почты, которые записываются в каждый коммит:

```
$ git config --global user.name "Иван Иванов"
$ git config --global user.email "ivan@example.com"
$ git config --global init.defaultBranch main
```

Основной цикл работы

Команда `uv init` создаёт git-репозиторий в каталоге проекта (при его отсутствии используется `git init`). Работа с репозиторием строится по циклу: изменение файлов → просмотр изменений → добавление изменений в индекс → создание коммита.

```
$ git status          # изменённые и неотслеживаемые файлы
$ git diff            # содержание изменений
$ git add stats.py     # добавление файла в следующий коммит
$ git commit -m "StationStats: update и merge"
$ git log --oneline    # история коммитов
```

Требования к истории изменений:

- каждый коммит содержит одно законченное по смыслу изменение (например, «разбор строки без `float`», «чтение файла блоками»); единственный коммит со всей работой не считается приемлемой историей;
- сообщение коммита кратко описывает сделанное изменение: `"StationStats: merge"`, а не `"fix"` или `"asdf"`;
- перед `git add` проверяется вывод `git status`, чтобы в коммит не попали лишние файлы.

Файлы, исключаемые из репозитория

Файл `.gitignore` содержит перечень путей, которые git не отслеживает; `uv init` создаёт его с `.venv` и кешами Python.

Публикация на GitHub

1. Регистрация на <https://github.com>.
2. Настройка доступа с компьютера. Наиболее простой способ — GitHub CLI: установить gh (<https://cli.github.com>) и выполнить gh auth login. Альтернативный способ — SSH-ключ: ssh-keygen -t ed25519, после чего публичный ключ ~/.ssh/id_ed25519.pub добавляется в Settings → SSH and GPG keys. Пароль от учётной записи для git push не используется.
3. Создание пустого репозитория (New repository, без README и .gitignore, поскольку они уже созданы локально). Репозиторий может быть публичным или приватным.
4. Связывание локального репозитория с удалённым и отправка коммитов:

```
$ git remote add origin git@github.com:<логин>/python-lab1.git
$ git push -u origin main          # первая отправка; далее достаточно git push
```

При использовании gh шаги 3–4 выполняются одной командой: gh repo create python-lab1 --private --source=. --push.

3 Синтаксис языка

3.1 Структура программы

```
def greet(name: str) -> str:
    return f"Привет, {name}!"

if __name__ == "__main__":
    for who in ["миг", "БГУ"]:
        print(greet(who))
```

Основные отличия от C++:

- **блоки кода задаются отступами**, а не фигурными скобками; двоеточие открывает блок. Стандартный отступ — 4 пробела;
- точка с запятой в конце инструкции не ставится, типы переменных не объявляются;
- name: str и -> str — *аннотации типов* (type hints). Интерпретатор их не проверяет: они служат документацией и используются редактором для подсказок. Проверку соответствия типов выполняют отдельные инструменты статического анализа (например, муру), которые рассматриваются в лабораторной работе №2;
- f"...{expr}..." — f-строка, аналог std::format;
- функция main не обязательна: код модуля выполняется сверху вниз. Условие __name__ == "__main__" истинно, только если файл запущен как скрипт, а не импортирован из другого модуля.

Стиль кода на Python описан в документе [PEP 8](#): snake_case для функций и переменных, CamelCase для классов, UPPER_CASE для констант.

3.2 Переменные, объекты и ссылки

В C++ переменная обозначает область памяти определённого типа. В Python переменная — это **имя, связанное с объектом**. Тип принадлежит объекту, а не имени, и одно имя может быть повторно связано с объектом другого типа.

```
x = 42          # имя x связано с объектом int
x = "строка"    # теперь -- с объектом str
a = [1, 2, 3]
b = a           # копирования нет: a и b -- два имени одного списка
b.append(4)
print(a)        # [1, 2, 3, 4]
c = a.copy()    # явная (поверхностная) копия
print(a == c, a is c) # True False: == сравнивает значения, is -- идентичность
```

Семантику переменных Python удобно представлять через `std::shared_ptr<Object>` в C++: присваивание копирует ссылку, а не объект. Память освобождается автоматически (подсчёт ссылок и сборщик циклов). Аргументы функций также передаются по ссылке.

Объекты делятся на **изменяемые** (`list`, `dict`, `set`, большинство пользовательских классов) и **неизменяемые** (`int`, `float`, `str`, `tuple`, `bytes`, `frozenset`). Содержимое неизменяемого объекта изменить нельзя — можно только создать новый объект:

```
s = "abc"
s += "d"      # создаётся новая строка "abcd", имя s связывается с ней
# s[0] = "X"  # TypeError: 'str' object does not support item assignment
```

3.3 Встроенные типы

Тип	Описание	Аналог в C++
<code>int</code>	целое произвольной длины, без переполнения	<code>long long</code> без границ
<code>float</code>	число с плавающей точкой двойной точности	<code>double</code>
<code>bool</code>	<code>True</code> / <code>False</code> (подкласс <code>int</code>)	<code>bool</code>
<code>str</code>	неизменяемая строка символов Unicode	<code>std::u32string</code>
<code>bytes</code>	неизменяемая последовательность байтов	<code>std::vector<uint8_t></code>
<code>None</code>	отсутствие значения, единственный объект своего типа	<code>nullptr</code> / <code>std::nullptr_t</code>
<code>list</code>	динамический массив ссылок на объекты	<code>std::vector<Object*></code>
<code>tuple</code>	неизменяемая последовательность фиксированной длины	<code>std::tuple</code>
<code>dict</code>	хеш-таблица, сохраняющая порядок вставки	<code>std::unordered_map</code>
<code>set</code>	хеш-множество	<code>std::unordered_set</code>

```
7 / 2          # 3.5    -- деление всегда даёт float
7 // 2         # 3      -- целочисленное деление с округлением вниз
-7 // 2        # -4     -- в C++ результат равен -3
-7 % 2         # 1      -- знак остатка совпадает со знаком делителя
2 ** 10        # 1024   -- возведение в степень
int("42"), float("3.14"), str(42) # преобразования типов
len("Минск")   # 5 символов
len("Минск".encode("utf-8"))      # 10 байт в кодировке UTF-8
```

3.4 Строки

```
s = "Hello, World"
s.lower(), s.upper()           # 'hello, world', 'HELLO, WORLD'
s.split(", ")                  # ['Hello', 'World']
"key=value".partition("=")     # ('key', '=', 'value')
" text \n".strip()             # 'text'
", ".join(["a", "b", "c"])     # 'a, b, c'
s.startswith("He"), "or" in s  # True True
s.find("o"), s.replace("l", "L")
f"{3.14159:.2f}", f"{42:>6}"    # '3.14', '      42'
```

Поскольку строки неизменяемы, операция `s += x` в цикле на каждой итерации создаёт новую строку. Для объединения большого числа фрагментов их накапливают в списке и объединяют вызовом `"".join(parts)`.

3.5 Коллекции

```
xs = [3, 1, 2]
xs.append(5)                    # добавление в конец, амортизированно O(1)
xs.pop()                       # удаление с конца, O(1)
xs.insert(0, 10)                # вставка, O(n)
xs.sort()                      # сортировка на месте
ys = sorted(xs, reverse=True)  # новый отсортированный список
xs[0], xs[-1]                  # первый и последний элементы
xs[1:3], xs[::-1]              # срезы [start:stop:step] создают копию части списка

point = (1.0, 2.0)             # кортеж
x, y = point                   # распаковка
a, b = b, a                    # обмен значений без временной переменной

ages = {"Анна": 19, "Борис": 20}
ages["Вера"] = 18              # вставка или перезапись
ages.get("Глеб")               # None, если ключ отсутствует (ages["Глеб"] -- KeyError)
ages.get("Глеб", 0)            # значение по умолчанию
"Анна" in ages                  # проверка наличия ключа, в среднем O(1)
for name, age in ages.items():
    print(name, age)

seen = {1, 2, 3}
seen.add(2)                    # множество не изменится
seen & {2, 5}, seen | {5}      # пересечение, объединение
```

Ключом словаря и элементом множества может быть только *хешируемый* объект — как правило, неизменяемый: `int`, `str`, `tuple` из хешируемых элементов. Список не может быть ключом словаря.

3.6 Управляющие конструкции

```
if x > 0 and not done:         # and, or, not вместо &&, ||, !
    ...                        # ... (или pass) -- пустое тело блока
elif x == 0:
    ...
else:
```

```

...

for i in range(10):          # 0..9; в общем виде range(start, stop, step)
    ...
for i, item in enumerate(items):  # индекс и элемент
    ...
for a, b in zip(xs, ys):      # параллельный обход двух последовательностей
    ...

while n > 1:
    n //= 2                  # операторов ++ и -- нет: n += 1

label = "чёт" if n % 2 == 0 else "нечет"    # условное выражение

match command.split():      # сопоставление с образцом (Python 3.10+)
    case ["go", direction]:
        move(direction)
    case ["quit" | "exit"]:
        stop()
    case _:
        print("неизвестная команда")

```

Цикл `for` в Python всегда перебирает элементы коллекции, аналогично `range`-based `for` в C++11; конструкции вида `for (int i = 0; i < n; ++i)` нет. Если помимо элемента требуется его индекс, используется `enumerate`, а не `range(len(xs))`.

3.7 Генераторы коллекций

Генераторы коллекций (comprehensions) — сокращённая запись построения коллекций:

```

squares = [x * x for x in range(10)]
evens   = [x for x in xs if x % 2 == 0]
lengths = {word: len(word) for word in words}    # словарь
letters = {ch for ch in "abracadabra"}          # множество
total   = sum(x * x for x in range(10))          # без промежуточного списка

```

3.8 Функции

```

def power(base: float, exp: int = 2) -> float:    # значение по умолчанию
    return base ** exp

power(3), power(3, 3), power(exp=3, base=2)      # позиционные и именованные аргументы

def mean(*values: float) -> float:                # произвольное число аргументов
    return sum(values) / len(values)

mean(1, 2, 3)

def connect(host: str, *, timeout: float = 1.0, **options) -> None:
    # аргументы после * передаются только по имени; **options -- словарь прочих
    print(host, timeout, options)

connect("bsu.by", timeout=5, retries=3)

def min_max(xs: List[int]) -> tuple[int, int]:    # возврат нескольких значений

```

```

    return min(xs), max(xs)

lo, hi = min_max([3, 1, 4])

```

Функции являются объектами: их можно присваивать переменным, передавать в другие функции и создавать в выражениях с помощью `lambda`:

```

words.sort(key=len)                # сортировка по длине
pairs.sort(key=lambda p: (p[1], p[0])) # по второму, затем по первому элементу

```

Значение аргумента по умолчанию вычисляется **один раз** — при определении функции. Изменяемое значение по умолчанию поэтому оказывается общим для всех вызовов:

```

def add(item, bucket=[]):          # ошибка
    bucket.append(item)
    return bucket

add(1); add(2)                     # второй вызов вернёт [1, 2]

def add(item, bucket=None):        # корректный вариант
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

```

3.9 Исключения

```

try:
    value = int(text)
except ValueError as e:
    print(f"не число: {e}")
    value = 0
else:
    print("разбор выполнен без ошибок") # если исключения не было
finally:
    print("выполняется всегда")

if count < 0:
    raise ValueError(f"count должен быть >= 0, получено {count}")

```

Часто используемые встроенные исключения: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `FileNotFoundError`, `ZeroDivisionError`. Пользовательские исключения наследуются от `Exception`.

В отличие от C++, где исключения принято использовать лишь для исключительных ситуаций, в Python они являются штатным механизмом управления потоком выполнения: например, цикл `for` завершается по исключению `StopIteration`. Стил «выполнить и обработать ошибку» (EAFP, *easier to ask for forgiveness than permission*) считается в Python нормой.

3.10 Работа с файлами

```

with open("data.txt", encoding="utf-8") as f:
    for line in f:
        print(line.rstrip("\n"))

```

текстовый режим
построчное чтение

```
# здесь файл уже закрыт, даже если внутри блока возникло исключение

with open("out.txt", "w", encoding="utf-8") as f:
    f.write("строка\n")

with open("data.txt", "rb") as f:
    chunk = f.read(1 << 20)
    text = chunk.decode("utf-8")
```

бинарный режим
чтение до 1 Миб -> bytes
bytes -> str

Конструкция with (контекстный менеджер) реализует идиому RAII: ресурс гарантированно освобождается при выходе из блока. В текстовом режиме байты файла декодируются в str; кодировку следует указывать явно (encoding="utf-8"), поскольку кодировка по умолчанию зависит от операционной системы.

3.11 Соответствие конструкций C++ и Python

C++	Python
std::vector<int> v{1, 2};	v = [1, 2]
v.push_back(3); v.size();	v.append(3); len(v)
std::unordered_map<K, V> m;	m = {}
if (m.count(k))	if k in m:
for (auto& x : v)	for x in v:
for (int i = 0; i < n; ++i)	for i in range(n):
a && b, a b, !a	a and b, a or b, not a
x ? a : b	a if x else b
nullptr	None
std::cout << x << '\n';	print(x)
std::getline(std::cin, s);	s = input()
std::stoi(s), std::to_string(x)	int(s), str(x)
// комментарий	# комментарий
throw std::runtime_error("...");	raise RuntimeError("...")
try {} catch (const E& e) {}	try: ... except E as e: ...

4 Задание

Работа состоит из двух частей. Результат сдаётся в виде git-репозитория на GitHub (раздел 2.4); преподавателю направляется ссылка на репозиторий. Тот же репозиторий используется в последующих лабораторных работах.

4.1 Часть 1. Создание проекта

1. Установить uv (раздел 2.3) и Python 3.13 (uv python install 3.13).
2. Создать проект командой uv init --python 3.13, скопировать в него каталог data/ из материалов работы и выполнить первый коммит.
3. Дополнить .gitignore так, чтобы в репозиторий не попадали каталог .venv и сгенерированные файлы с данными.
4. Создать репозиторий на GitHub (публичный либо приватный с доступом для преподавателя) и отправить в него проект. В ходе дальнейшей работы изменения фиксируются

отдельными коммитами и регулярно отправляются командой `git push`.

5. Итоговая структура проекта:

```
lab1/
├── .gitignore
├── .python-version
├── pyproject.toml
├── uv.lock
├── data/                # материалы работы (генератор, примеры, check.py)
├── README.md           # инструкция по запуску
└── main.py             # решение задачи из части 2
```

6. Все функции снабжаются аннотациями типов; имена соответствуют PEP 8.

4.2 Часть 2. Задача «Миллиард строк»

В 2024 году в Java-сообществе проводился конкурс [One Billion Row Challenge](#): требовалось как можно быстрее обработать текстовый файл из миллиарда строк с измерениями температуры. Задача проста по формулировке, однако при таком объёме данных каждая лишняя операция в цикле обработки обходится в секунды. В данной работе требуется корректное решение задачи на Python. Решение может быть любой сложности, в том числе с использованием многопоточности и различных механизмов операционной системы, но без сторонних библиотек. Данные темы также будут затронуты только в последующих лабораторных работах.

Входные данные

f Текстовый файл в кодировке UTF-8; каждая строка имеет вид <станция>;<температура>:

```
Minsk;7.3
Hamburg;12.0
Zürich;-3.4
Bulawayo;8.9
Palembang;38.8
```

Гарантируется, что:

- название станции — непустая строка длиной не более 100 байт в UTF-8, не содержащая символов `;` и `\n`;
- температура — число от -99.9 до 99.9 ровно с одним знаком после точки;
- число различных станций не превышает 10 000;
- каждая строка, включая последнюю, завершается символом `\n`.

Выходные данные

Для каждой станции вычисляются минимальная, средняя и максимальная температура. Результат выводится по одной строке на станцию в формате <станция>;<min>;<mean>;<max>:

```
Abha;-0.8;18.3;48.0
Abidjan;-5.2;24.4;54.4
...
Zürich;-22.6;9.8;36.6
```

```
İzmir;-7.3;17.2;44.7
```

- станции упорядочены по названию встроенной функцией `sorted` (сравнение строк по кодам символов Unicode, поэтому, например, `İzmir` следует после `Zürich`);
- каждое число выводится ровно с одним знаком после точки;
- среднее значение округляется до десятых по правилу «половина вверх»: $\lfloor 10 \cdot \text{mean} + 0.5 \rfloor / 10$, т. е. $1.25 \rightarrow 1.3$, $-1.25 \rightarrow -1.2$.

Материалы

Каталог `data/` содержит:

- `generate_measurements.py` — генератор входных файлов. Файлы объёмом в миллионы строк в репозиторий не добавляются и генерируются локально:

```
$ uv run data/generate_measurements.py 10_000_000 -o measurements_10m.txt
```

Генератор детерминирован: при одинаковых значениях `N` и `--seed` создаётся один и тот же файл. Ориентировочный размер: 10 млн строк — около 140 МБ, 100 млн — 1,4 ГБ, 1 млрд — 14 ГБ;

- `weather_stations.csv` — список станций со среднегодовыми температурами, используемый генератором;
- `measurements_1k.txt`, `measurements_100k.txt` — небольшие входные файлы для отладки; `*.out` — эталонные результаты для них;
- `check.py` — программа проверки результата:

```
$ uv run main.py data/measurements_100k.txt \
  | python data/check.py data/measurements_100k.out
OK
```

Требования к решению

1. **Программа** `main.py` принимает путь к файлу и выводит результат:

```
$ uv run main.py measurements_10m.txt
Abha;-0.8;18.3;48.0
Abidjan;-5.2;24.4;54.4
...
```

2. **Объём данных.** Программа обрабатывает файл из 100 млн строк (около 1,4 ГБ); объём используемой памяти не зависит от размера входного файла.
3. **Ошибки.** При отсутствии аргументов или несуществующем файле программа выводит сообщение об ошибке в `stderr` и завершается с ненулевым кодом возврата, без вывода трассировки стека.
4. **Корректность.** Результат совпадает с эталоном на файлах `measurements_1k` и `measurements_100k` (`check.py` выводит `OK`).

5 Порядок сдачи

До защиты преподавателю направляется ссылка на репозиторий GitHub. Доступность ссылки проверяется без входа в учётную запись (например, в приватном окне браузера); для приватного репозитория проверяется наличие преподавателя в списке Collaborators. Сданное состояние репозитория помечается тегом `lab1`; проверке подлежит именно оно:

```
$ git tag lab1
$ git push origin lab1
```

На защите студент:

1. демонстрирует репозиторий и историю коммитов; в репозитории отсутствуют `.venv` и файлы с данными большого объёма, присутствует `uv.lock`;
2. клонирует репозиторий в пустой каталог и запускает программу командой `uv run` без дополнительной настройки;
3. запускает `check.py` на файле `measurements_100k`;
4. отвечает на вопросы по коду.

Возможные вопросы

1. Что такое коммит? Каково назначение команд `git add`, `git commit`, `git push`? Для чего нужен файл `.gitignore`?
2. Для чего используются виртуальные окружения? Что содержит каталог `.venv` и почему он не добавляется в репозиторий? Чем `uv.lock` отличается от списка зависимостей в `pyproject.toml`?
3. Каков результат выполнения `b = a; b.append(1)`, если `a` — список? Каков результат `b = a; b += 1`, если `a` — число? Чем объясняется различие?
4. Чем различаются операторы `==` и `is`? В каких случаях уместно `is`?
5. Какие объекты могут быть ключами словаря и почему?
6. В чём состоит проблема изменяемого значения аргумента по умолчанию? Как её избежать?
7. Проверяет ли интерпретатор аннотации типов во время выполнения? Для чего они используются?
8. Чем различаются текстовый и бинарный режимы открытия файла? Каким образом в решении организовано чтение файла?
9. Каким образом в решении обеспечено округление среднего значения по заданному правилу?

Источники

- Официальное руководство по Python: <https://docs.python.org/3.13/tutorial/>
- Документация uv: <https://docs.astral.sh/uv/>
- PEP 8 — руководство по стилю кода: <https://peps.python.org/pep-0008/>
- Pro Git (книга о git, есть перевод на русский язык): <https://git-scm.com/book/ru/v2>
- One Billion Row Challenge: <https://github.com/gunnarmorling/1brc>